

TP 2.2 Git : dépôt distant

Objectifs du TP

- L'utilisation de Git et GitHub
- L'utilisation de GitHub avec Visual Studio Code

Git et GitHub

Git est un logiciel de versioning, ou logiciel de gestion de version qui permet de conserver un historique des modifications effectuées sur un projet afin de pouvoir rapidement identifier les changements effectués et de revenir à une ancienne version en cas de problème.

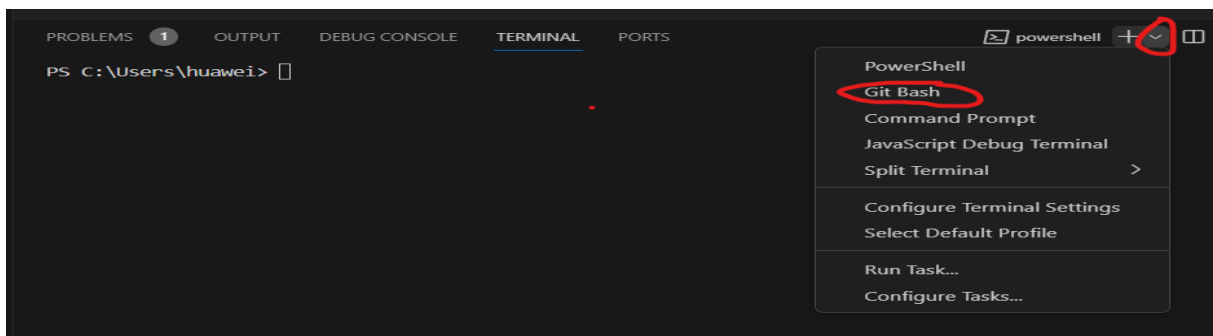
GitHub est un service basé sur le cloud qui héberge le système de contrôle de version (VCS) Git. Il permet aux développeurs de collaborer et d'apporter des modifications aux projets partagés tout en gardant un suivi détaillé de leur progression.

1. Création d'un compte GitHub (si vous n'en avez pas déjà un)

Si vous n'avez pas un compte, allez sur GitHub. Saisissez un nom d'utilisateur(username), une adresse e-mail et un mot de passe pour votre compte. Pour valider votre compte, accédez à votre boîte e-mail. Vous trouverez un e-mail expliquant comment valider le compte nouvellement créé. Vérifiez section spam, au cas où vous n'auriez rien reçu dans votre inbox.

2. Travailler avec Git Bash dans Visual Studio Code

Dans la barre supérieure de Visual Studio Code., cliquez sur Terminal pour ouvrir un nouveau terminal. Puis, choisir Git Bash :



3. Réglages de base pour Git

La première étape pour utiliser Git est de configurer votre nom et votre adresse e-mail avec les commandes suivantes :

```
$git config --global user.name "Prénom nom"
```

```
$git config --global user.email "adresse_email"
```

4. Créer un repository local

Dans le répertoire personnel, nous allons créer un répertoire dans lequel on va initialiser le repository Git.

- a. Créer un projet « TP2 » dans le dossier « C:\git » et initialiser le dépôt :

```
$ cd C:\git
```

```
$ mkdir TP2
```

```
$ cd TP2
```

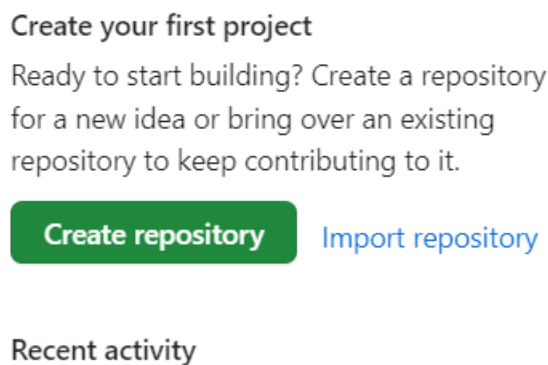
```
$ git init
```

- b. Ajouter le fichier «Readme.md » au répertoire de travail
- c. Indexer ce fichier avec \$ git add
- d. Réaliser un commit: \$ git commit -m "first commit"
- e. Afficher l'historique des modifications.

5. Créer un repository sur GitHub

Nous allons créer un repository sur GitHub, et après nous l'interconnecterons avec le repository local :

- a. La première étape consiste à se connecter à GitHub.
- b. Ensuite, cliquez sur « create repository » :



- c. Donner le nom TP2 au projet, une brève description de celui-ci et vous pouvez décider s'il doit être public (visible par tous les utilisateurs) ou privé (visible uniquement par nous et les collaborateurs potentiels du projet). Pour ce TP, nous allons créer un référentiel public.

d. Cliquez sur « Create repository ».

Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere?
[Import a repository.](#)

Required fields are marked with an asterisk (*).

Owner *  ramzi398 / Repository name *

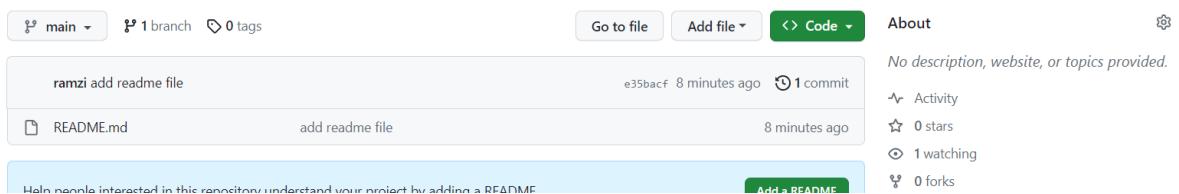
TP2 is available.

Great repository names are short and memorable. Need inspiration? How about [automatic-funicular](#) ?

Description (optional)

- ☒  **Public**
Anyone on the internet can see this repository. You choose who can commit.
- ☐  **Private**
You choose who can see and commit to this repository.

- e. Associer le dépôt distant et le dépôt local, taper cette commande sur git bash
\$ git remote add origin <https://github.com/<utilisateur>/TP2.git>
- f. Renommer la branche master en main : \$git branch -M main
Lister les branches avec la commande : \$ git branch -r
- g. Synchroniser les deux emplacements : "Pousser" les changements de local vers github
(remote repository) : \$git push -u origin main
- h. Vérifier la synchronisation sur l'interface web de GitHub.

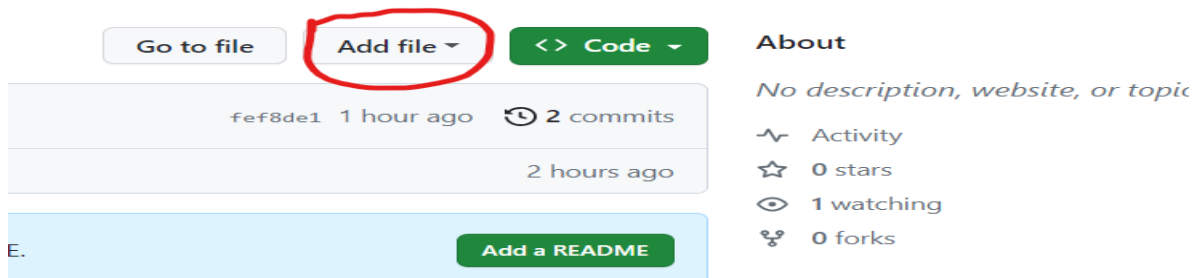


- i. Modifie le fichier README.md sur le dépôt local en ajoutant le texte ci-dessous et valider les changements par un commit.

Bienvenue

Programme Java qui affiche le nom de l'étudiant en utilisant la classe SimpleTesting ()`.

- j. Publier les modifications sur le dépôt distant. Vérifier sur l'interface web de GitHub.
- k. Ajouter le fichier « SimpleTesting.java » directement à partir l'interface web de



GitHub :

- l. Ajouter le code ci-dessous à la classe Java :

```
public class SimpleTesting {
    public static void main(String[] args) {
        Student student = new Student("John");
        String name = student.getName();
        System.out.println("Student name is : "+name);
    }
}
```

m. Mettre à jour le dépôt local : prendre les changements de remote repository avec :

```
$ git pull origin main
```

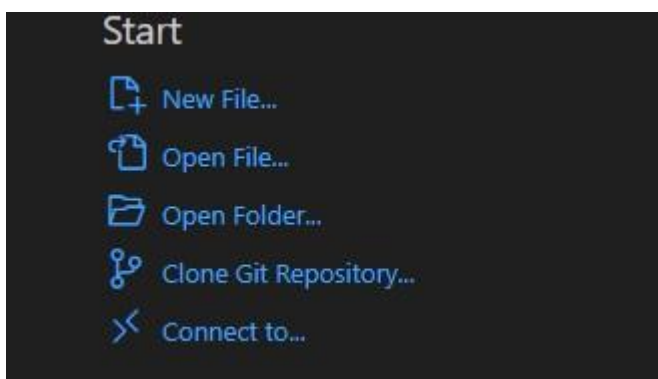
n. Vérifier dans votre dossier de dépôt local.

6. Travail collaboratif

Dans cette partie, on travaille par binôme, et on se sert de Git comme outil d'archivage de la version stable et comme outil de partage et de distribution des sources.

Pour apporter en Visual Studio Code, le repository créé dans la première partie du TP, nous utiliserons la commande **git clone** suivie par le lien vers repository :

- Créez un dossier avec le nom *TP2-vsc* et ouvrez-le in Visual Studio Code (Ensuite, nous utiliserons le terminal de Visual Studio Code (git bash)).
- Cliquer sur « clone repository » et ajouter l'URL <https://github.com/<utilisateur>/TP2.git> (même URL pour les 2 étudiants, choisir un seul compte)



Ou avec le terminal Git bash, écrire la commande :

```
$git clone https://github.com/<utilisateur>/TP2.git
```

Etudiant 1 :

Créer un fichier README1.md et écrire dans ce fichier le texte `"# VSC-TP1_NomEt1"`, et modifier la classe java « my name is » dans le message à afficher par `System.out.println`

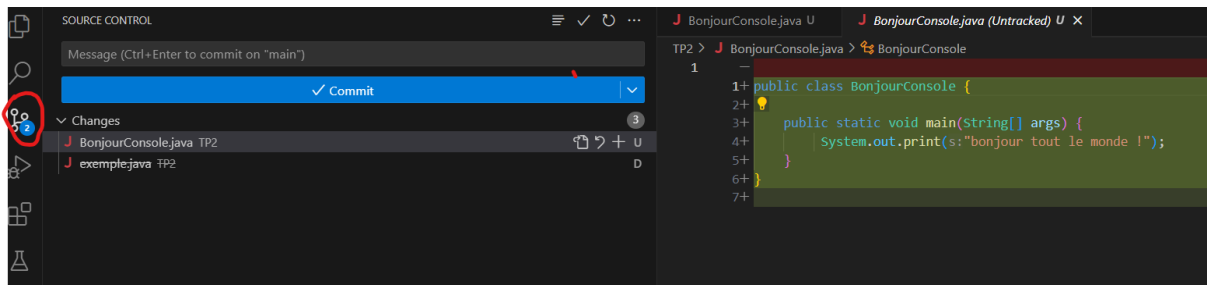
Etudiant 2 :

Ecrire dans le fichier README.md le texte ”# VSC-TP1_NomEt2”, et Créer une classe java « Student » et ajouter ce code

```
class Student{  
    String name;  
    Student(String name){  
        this.name = name;  
    }  
    public String getName() {  
        return this.name;  
    }  
}
```

Etudiants 1&2 :

Après les modifications, vous pouvez voir que sur le côté gauche de l'icône de “Source Control” apparaît le numéro 2, qui représente le nombre de modifications. Si vous cliquez dessus, vous verrez une liste des changements, appuyez sur l'un d'eux et VSCode affichera deux versions du fichier modifié et l'original.

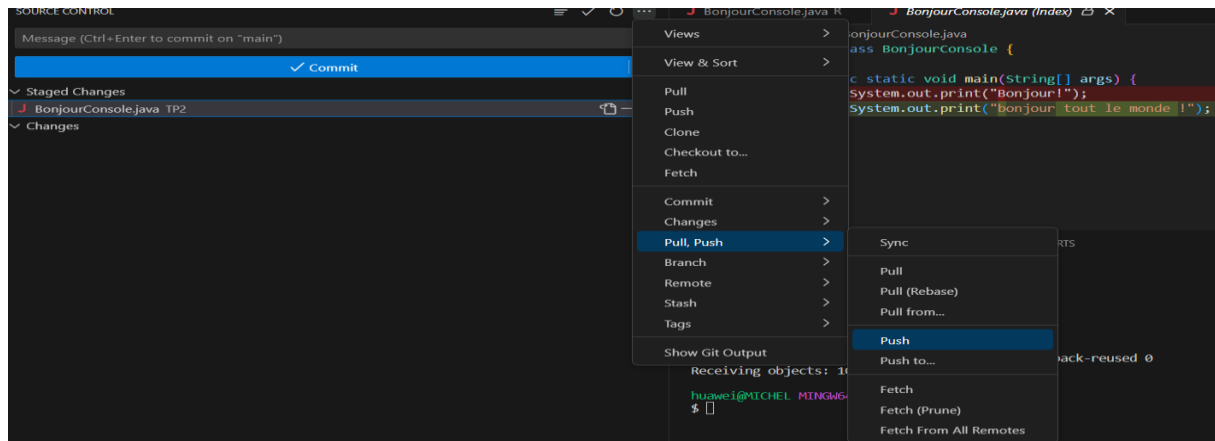


Pour enregistrer la nouvelle version du projet et la mettre sur GitHub, nous devons faire commit, puis push. Nous pouvons utiliser les commandes apprises dans la première partie, ou aussi avec le gestionnaire de contrôle de source Git sur VSCode :

Commit : écrire un message de commit « first_commit » puis cliquer sur commit, ou indexer les fichiers et valider avec Git bash : `$git commit -m « first_commit »`.

Push : Pour faire “push”, nous cliquons sur “More actions”, les 3 points dans le coin, puis un onglet s'ouvre où nous pouvons voir plus de commandes. Cliquer sur “Push”, et les modifications seront envoyés sur GitHub.

Vérifier sur l'interface web de GitHub.



Après que les deux étudiants envoient leurs travaux, il faut mettre à jour le dépôt local : prendre les changements de remote repository. Cliquer sur « Pull ».

Etudiant 1 :

Modifier la classe SimpleTesting en mettant le nom de votre binôme dans la ligne 3

Student student = new Student("Nom_E2");

Valider le commit par un message « changement de nom ET2 »

Puis « push »

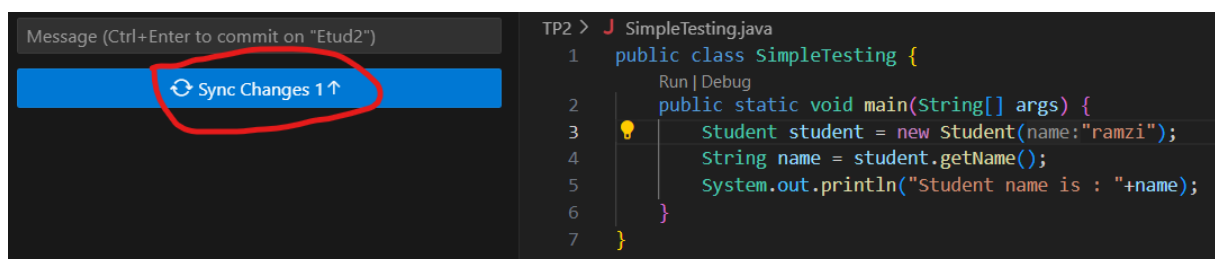
Etudiant 2 :

Modifier la classe SimpleTesting en mettant le nom de votre binôme dans la ligne 3

Student student = new Student("Nom_E1");

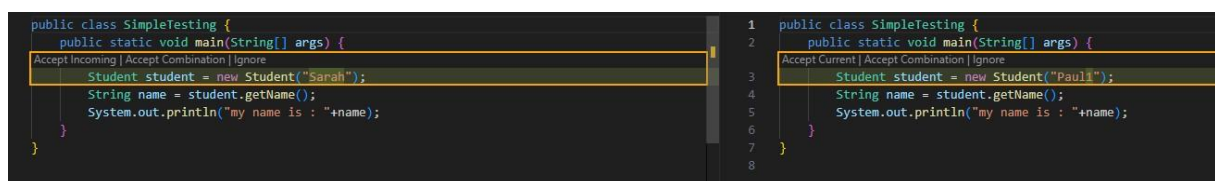
Valider le commit par un message « changement de nom ET1 »

Synchroniser les deux travaux



Qu'est-ce que vous remarquez ? est-ce que la fusion a été bien faite ? dans quel fichier on a le problème de fusion ?

Résoudre le problème en choisissant l'un des noms.

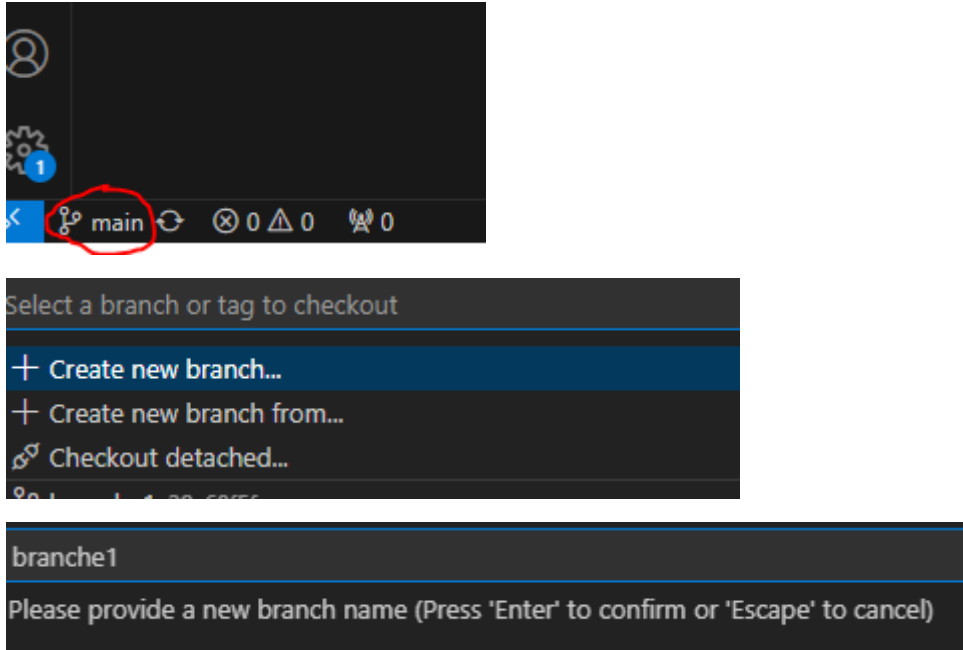


Valider par un commit, puis Synchroniser.

Exécuter le programme Java.

7. Branches dans un dépôt distant

Créer une nouvelle branche 'Branche1' : cliquer sur « main » puis « create new branch » et donner le nom « Branche1 » à cette branche (Branche 2 pour Etudiant2)



Publier cette branche

Lister les branches : `$git branch -r`

Basculer à cette branche : cliquer sur « main » puis choisir le nom de la branche dans la liste

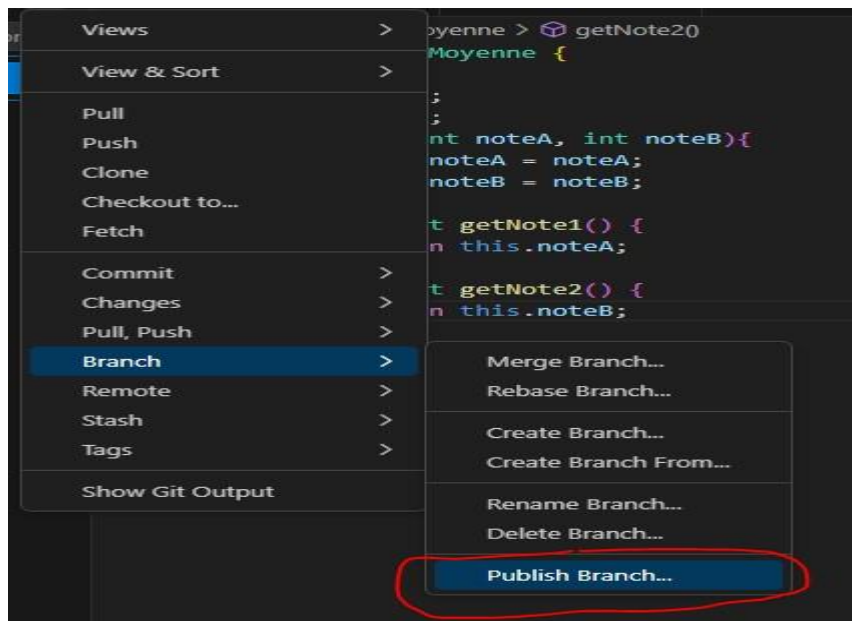
Ou dans git bash avec la commande : `$git checkout Branche1`

Créer une classe « Moyenne.java » qui permet de calculer la moyenne de deux notes

Modifier la classe « StudentTesting.java » pour afficher la moyenne de l'étudiant donné.

Indexer et valider par un commit : `$git commit -a -m « ajout de la classe Moyenne »`

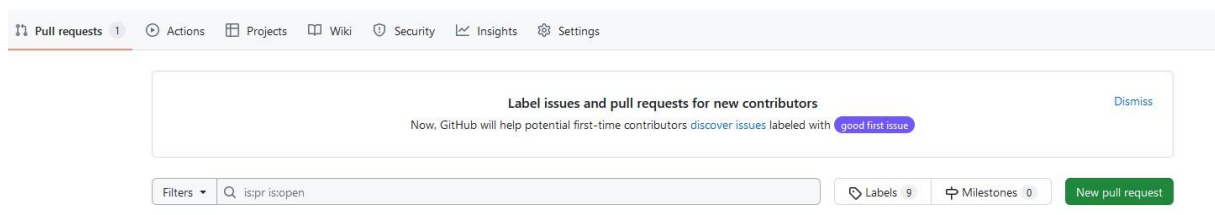
Publier cette branche



Maintenant sur l'interface GitHub : cliquer sur « branches » pour voir la branche



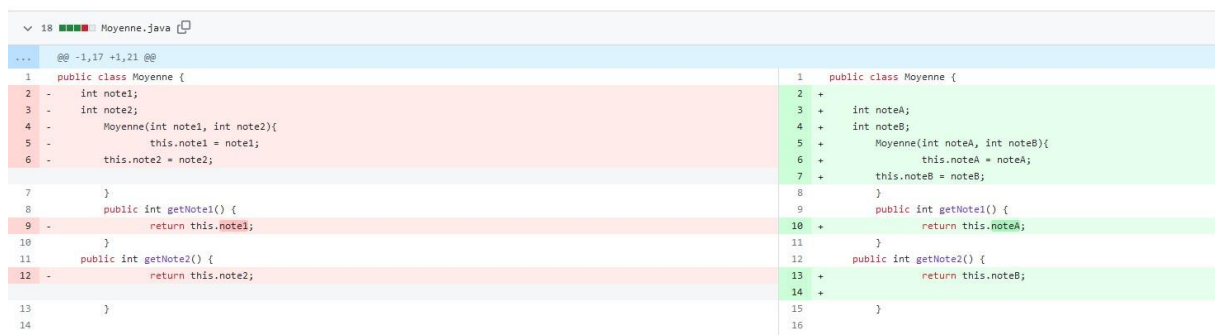
Cliquer sur « Pull requests », puis sur « new pull request »



Cliquer sur « Merge »

Qu'est-ce que vous remarquez ? est-ce que la fusion a été bien faite ?

Comparer les deux branches



Choisir une classe puis valider la fusion.